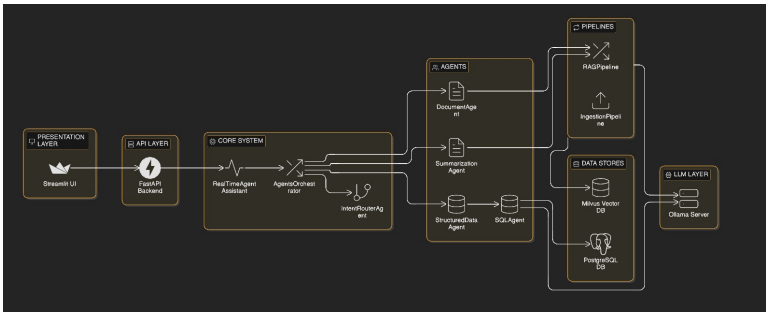
System Design Document

Agentic RAG System

1. System Architecture Diagram

The system architecture encompasses a local frontend, a FastAPI backend, vector and relational databases, and a local LLM orchestrator.

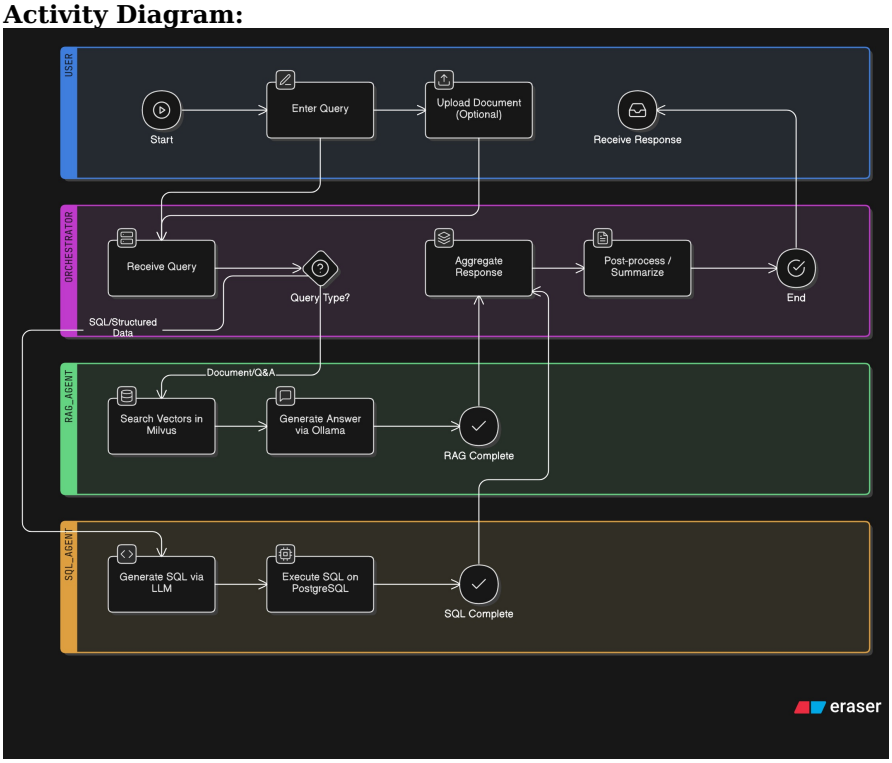


Component Diagram (Full System Architecture)

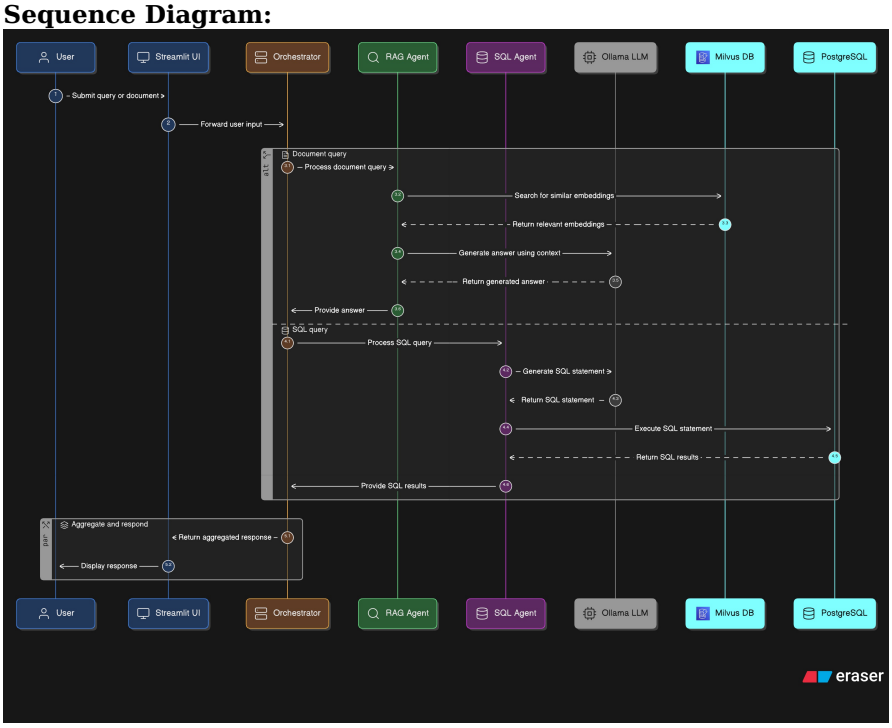
2. Agentic Workflow Design

The Agentic RAG System employs a dynamic routing mechanism to handle different types of queries: - **Intents:** The `IntentRouterAgent` classifies user queries into `document_qa`, `structured_data`, or `summarize`. - **Heuristics:** Parallel to the LLM router, a heuristic-based SQL detector ensures deterministic capture of straightforward data queries. - **Orchestration:** The `AgentsOrchestrator` invokes the appropriate agent based on the classification and aggregates the response, incorporating retry mechanisms and fallbacks.

Activity Diagram:



Sequence Diagram:



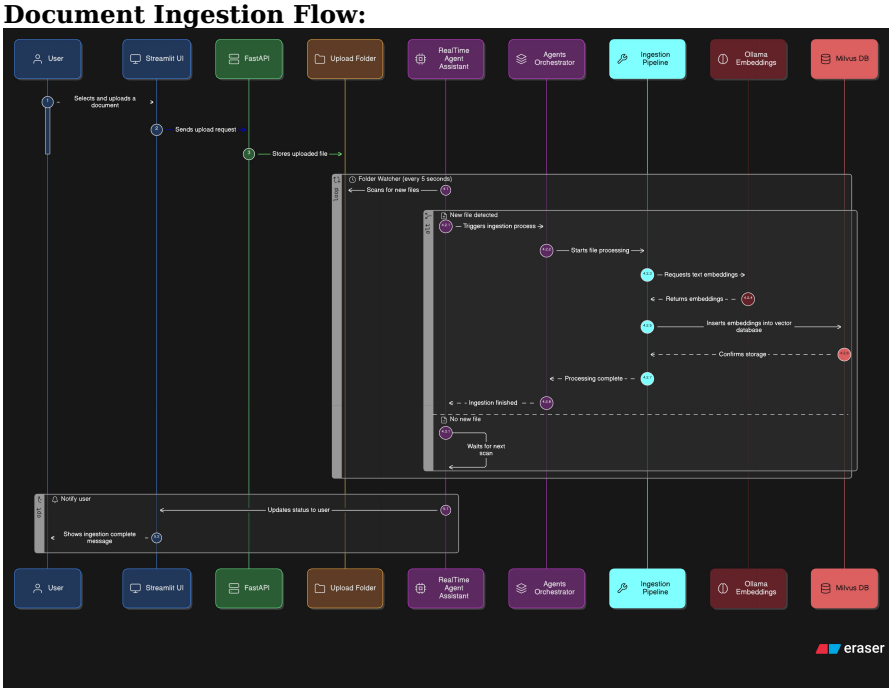
3. Context Construction Strategy

Context is built dynamically depending on the detected intent: 1. **Unstructured Data (Documents):** A two-stage retrieval pipeline is used (`rag_pipeline.py`). - First, an Approximate Nearest Neighbor (ANN) search retrieves the top 20 candidate chunks from Milvus

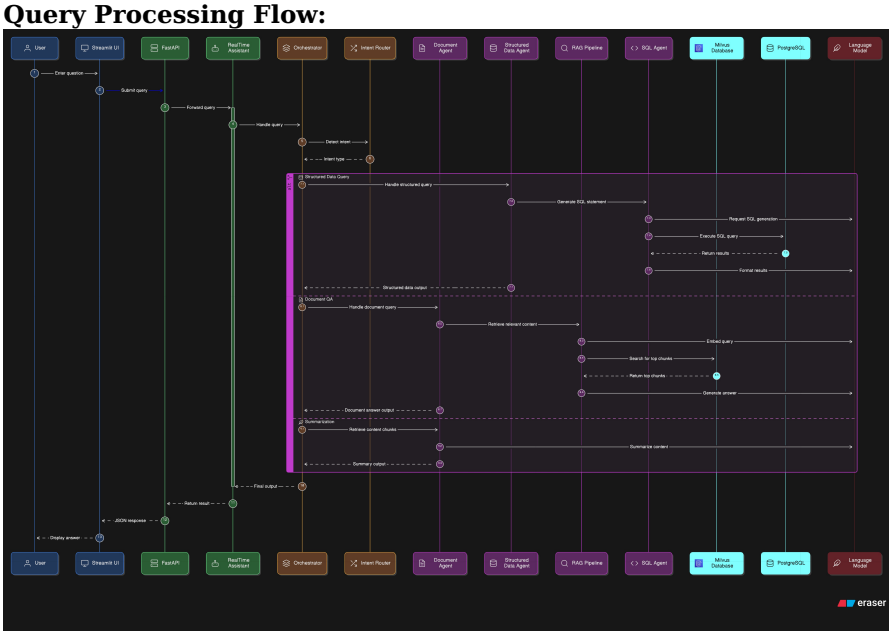
using dense vector embeddings. - Second, a hybrid re-ranking step merges cosine similarity scores with keyword overlap scores to filter and order the top 8 chunks. - These chunks form the context window provided to the LLM. 2. **Structured Data (CSV/Excel):** The `SQLAgent` constructs context by generating a PostgreSQL query representing the user's question. The raw database output is then fed into the LLM alongside the original query to format a natural language response.

4. Document Ingestion & Query Processing Flows

Document Ingestion Flow:



Query Processing Flow:



5. Technology Choices and Rationale

Component	Technology	Rationale
API Framework	FastAPI	Lightweight, asynchronous, high-performance web framework. Excellent for building fastREST APIs.
Vector Database	Milvus	Highly scalable vector database, supporting fast ANN searches and hybrid filtering strategies.
SQL Database	PostgreSQL	Robust relational database to store CSV/Excel tabular data reliably.
LLM & Embeddings	Ollama (gemma3n)	Runs local open-source models (Gemma) ensuring privacy, zero API costs, and low latency.
UI	Streamlit	Rapid development of interactive chat interfaces and file upload widgets.
Parsing	LlamaIndex	Offers powerful `SimpleDirectoryReader` capable of handling diverse formats (PDF, DOCX, PPTX).

6. Key Design Decisions

- **Segregation of Data Types:** Unstructured data is routed to Milvus via a chunking-embedding pipeline, while structured files (CSV/XLSX) bypass the vector store and are instantiated as native PostgreSQL tables. This prevents tabular data from losing its relational structure.
- **Dual-Path Intent Routing:** Relying solely on an LLM for classification can introduce latency and variability. Combining the LLM router with a regex/heuristic SQL fallback provides robustness and speed.
- **Two-Stage Retrieval:** Instead of dumping raw top-K vector search results into the LLM, the system over-fetches (top 20) and re-ranks based on exact keyword overlap. This significantly reduces LLM hallucination caused by superficially similar but factually irrelevant vectors.

7. Limitations

- **Read-Only SQL Operations:** For safety, the SQL agent enforces a strict regex allowing only `SELECT` and `WITH` statements. System data cannot be modified via natural language.
- **Local LLM Constraints:** While Ollama provides privacy, the reasoning capabilities of local 7B-9B models may fall short of large frontier models for highly complex multi-hop reasoning tasks.
- **Resource Intensive:** Running parallel embeddings and local LLM inferences requires substantial RAM/VRAM. Large file uploads may briefly spike host CPU/Memory usage.
- **No Multi-tenancy/Auth:** The system is currently designed as a single-user local assistant. Uploaded documents are pooled into a single Milvus collection, and there are no user permission boundaries.